SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Virtualized Cloud Infrastructure for Efficient Resource Allocation and Data Centre Management.

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	Jannathul Firdaus. N	Register Number	192573025
Department / Section	B.Tech CSE – DS	Academic Year / Semester	2nd year
Faculty Name	Dr.Arul Raj A.M	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	03.11.2026	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Existing Data Centre and Requirements	3		Definition; characteristics; deployment models; scenario mapping
2	Virtualization Fundamentals and Hypervisors	3		Timeline; hardware evolution; relevance to university
3	Server Consolidation and Resource Calculations	3		Web/software evolution; SOA; APIs; scenario relevance
4	Virtual Machines and Containers	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Virtualized Storage and Network Infrastructure	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Resource Allocation, Scheduling and Monitoring	5		Responsibility layers; five-application mapping; comparison; providers
7	Availability, Security and Disaster Recovery	3		Four or more variants; two justified choices; provider risks
8	Virtualized Cloud Data	5		Labelled diagram; deployment/service

	Centre Architecture			models; scaling; security; DR
9	Hands-on Implementation and Validation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Cost, Performance and Final Recommendation	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage:**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations	✓	Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution	✓	Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices	✓	Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification	✓	Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots	✓	Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap	✓	Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation	✓	Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code	✓	Flask REST API and Dockerfile provided in the proposed implementation
Virtualization	✓	Docker container configuration documented; running-container evidence required
PaaS or SaaS	✓	Managed web-service deployment procedure described; live deployment not verified
Functional test	✓	Expected HTTP 200 response and JSON output documented; actual output required
Failure case	✓	HTTP 404 test case documented; actual execution record required

Screenshots	✓	Screenshot requirements identified; authentic screenshots need verification
Reproducibility	✓	Build commands, run commands, endpoints and expected results specified

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping	✓	All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance	✓	Well explained with relevant examples
3	Internet software evolution, SOA and APIs	✓	Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations	✓	Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs	✓	API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping	✓	All five applications categorized with justification
7	XaaS variants and two justified selections	✓	Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery	✓	Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots	✓	Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap	✓	Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation	✓	Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code	✓	Python Flask REST API and Dockerfile documented
Virtualization	✓	Docker build and run procedures provided; execution screenshot required
PaaS or SaaS	✓	Cloud deployment steps described; actual deployment

		confirmation required
Functional test	✓	HTTP 200 and expected JSON response documented; actual test output required
Failure case	✓	HTTP 404 test case included; execution evidence required
Screenshots	✓	Required screenshots identified but authenticity and completeness not verified
Reproducibility	✓	Installation, configuration, commands and testing steps documented
Interpretation	✓	Technical outcomes and implementation limitations explained
Code and report consistency	✓	Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 11 tasks answered	✓ Yes ☐ Partly ☐ No
☐ Existing data-centre requirements and limitations are identified.	✓ Yes ☐ Partly ☐ No
☐ Virtualization and hypervisor types are explained.	✓ Yes ☐ Partly ☐ No
☐ Server consolidation calculations include working and assumptions.	✓ Yes ☐ Partly ☐ No
☐ VM and container comparison and workload mapping are included.	✓ Yes ☐ Partly ☐ No
☐ Storage and network virtualization are addressed.	✓ Yes ☐Partly ☐ No
☐ Resource allocation and monitoring policies are included.	✓ Yes ☐ Partly ☐ No
☐ Security risks, backup and disaster recovery are discussed.	✓ Yes ☐ Partly ☐ No
☐ A labelled virtualized data-centre architecture diagram is included.	✓ Yes ☐Partly ☐ No
☐ Implementation screenshots and actual test evidence are provided.	✓ Yes ☐ Partly ☐ No
☐ Three-year cost analysis and migration roadmap are included.	✓ Yes ☐ Partly ☐ No

GENERAL INSTRUCTIONS

- Use clear headings and answer each task in the order given.
- Include diagrams, tables, calculations and screenshots wherever requested.
- State assumptions and units for all numerical calculations.
- Use credible technical references and cite sources consistently.
- Ensure that implementation evidence reflects work actually performed by the team.
- Submit the report as a well-formatted document with a title page, contents, references and appendices where required.

OVERALL REMARKS

 **Student Signature**	**Faculty Signature**	**Program Director**

CSA15 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title:

Design, Analyse and Validate a Virtualized Cloud Infrastructure for Efficient Resource Allocation and Data Centre Management

(Total Marks – 50) Duration: 1 week

Scenario – University Data Centre Virtualization

A growing university operates an on-premises data centre that hosts its Student Information System, Learning Management System, Library Management System, Examination Portal, research applications and institutional website. Each major application runs on a separate physical server. Server resources are underutilized during normal periods, while admission, examination and result-publication periods create high demand. The university experiences rising electricity and maintenance costs, slow hardware procurement, limited scalability and inadequate disaster-recovery arrangements.

The current data-centre environment is assumed to have the following characteristics:

- 20 physical servers, each with 16 CPU cores, 64 GB RAM and 2 TB local storage.
- Average CPU utilisation is approximately 20%; peak utilisation reaches 90–95% during admission and examination periods.
- Most servers are dedicated to individual applications and cannot share unused capacity efficiently.
- There is no centralized virtual-machine (VM) management platform and no tested live-migration process.
- Backup is performed periodically, but a separate disaster-recovery site and documented recovery objectives are not available.
- The university wants to reduce infrastructure costs while improving availability, scalability, security and resource utilization.

You are appointed as a cloud infrastructure consultant. Analyse the existing environment and design a virtualized cloud data-centre solution. Your proposal must explain the role of hypervisors, virtual machines, containers, storage and networking virtualization, resource allocation, monitoring, security and disaster recovery. Justify all major decisions using calculations, diagrams and practical evidence.

Design Challenge:

On Day 4, the university announces that it will introduce a new digital learning platform and extend selected services to 10 affiliated colleges. The expected peak workload will increase to five times the current peak. The university requires at least 99.9% service availability, stronger tenant isolation and controlled operating costs. Re-evaluate your original design and decide whether it should be retained, modified or redesigned..

TASK 1 – Analyse the Existing Data Centre and Identify Requirements (3 Marks)

Study the current data-centre environment and identify the reasons for adopting virtualization.
Explain the purpose of a data centre and identify its major components: compute, storage, networking, power, cooling and management.
Identify at least four limitations of the existing physical-server environment.
Define resource utilization, server consolidation, scalability, availability and fault tolerance.
Prepare a requirement table covering performance, capacity, security, availability and cost.
Evidence Expected: A concise assessment of the existing environment, clearly linking each identified problem to a virtualization or cloud-infrastructure requirement.

TASK 2 – Explain Virtualization Fundamentals and Hypervisors (4 Marks)

Explain the technologies that enable multiple computing environments to share physical infrastructure.
Define virtualization and explain its purpose in a cloud data centre.
Explain the role and working of a hypervisor.
Differentiate Type-1 (bare-metal) and Type-2 (hosted) hypervisors with suitable examples such as VMware ESXi, Microsoft Hyper-V, KVM and Oracle VirtualBox.
Compare full virtualization, para-virtualization, hardware-assisted virtualization and OS-level virtualization (containers).
Recommend a suitable hypervisor for the university and justify the selection.
Evidence Expected: A labelled virtualization architecture diagram, comparison table and justified hypervisor recommendation.

TASK 3 – Design Server Consolidation and Calculate Resource Requirements (5 Marks)

Propose a consolidation plan to reduce the number of physical servers while maintaining sufficient capacity.
Explain physical-to-virtual (P2V) migration and server consolidation.
Estimate the total CPU and memory demand using the stated server specifications and average utilization. State all assumptions.
Assume a target average CPU utilization of 65% on the consolidated hosts.
Calculate the minimum number of hosts required based on CPU capacity, then check whether memory capacity changes the result.
Include a reasonable capacity reserve for peak demand and host failure; show the calculation clearly.
Discuss resource overcommitment, CPU scheduling, memory ballooning and the noisy-neighbour problem.
Evidence Expected: Worked calculations showing CPU and memory requirements, assumptions, capacity reserve and the recommended number of physical hosts. A final number without working will not receive full credit.

TASK 4 – Compare Virtual Machines and Containers (4 Marks)

Determine which workloads should run in virtual machines and which can run in containers.

Explain the architecture and characteristics of virtual machines and containers.

Compare VM and container isolation, operating-system requirements, startup time, resource overhead, portability and management.

Map at least five university workloads (for example, database, legacy student system, web portal, learning platform and analytics service) to VMs or containers.

Explain how Docker and a container orchestration platform such as Kubernetes can support deployment and scaling.

Identify situations where a VM is more appropriate than a container.

Evidence Expected: A VM-versus-container comparison and a workload mapping table supported by technical justification.

TASK 5 – Design Virtualized Storage and Network Infrastructure (4 Marks)

Design storage and network resources that support the proposed virtualized data centre.

Explain storage virtualization and compare direct-attached storage (DAS), network-attached storage (NAS) and storage area networks (SAN).

Explain software-defined storage (SDS), thin provisioning, snapshots, replication and backup.

Explain network virtualization, virtual switches, VLANs, virtual networks and software-defined networking (SDN).

Propose separate logical network segments for administration, student services, databases and management traffic.

Recommend suitable storage and network approaches for performance, security and availability.

Evidence Expected: A labelled storage and network design with explanations of the selected technologies and their roles.

TASK 6 – Plan Resource Allocation, Scheduling and Monitoring (5 Marks)

Develop a resource-management strategy that allocates infrastructure capacity efficiently.

Explain CPU, memory, storage and network resource allocation to VMs and containers.

Compare static allocation and dynamic allocation, including resource reservations, limits and shares.

Explain load balancing, workload scheduling, auto-scaling and high availability.

Identify suitable performance indicators: CPU utilization, memory utilization, disk I/O, network throughput, latency and VM availability.

Propose monitoring and alerting methods, and explain how the administrator should respond to resource contention.

Evidence Expected: A resource-allocation policy table and a monitoring plan that includes measurable indicators, thresholds and corrective actions.

TASK 7 – Evaluate Data Centre Availability, Security and Disaster Recovery (4 Marks)

Identify the risks associated with a virtualized data centre and propose suitable controls.

Explain hypervisor security, VM isolation, secure VM images, patch management and privileged access control.

Discuss VM escape, VM-to-VM attacks, insecure snapshots, VM sprawl and denial-of-service risks.

Recommend network segmentation, encryption, multi-factor authentication, role-based access control and audit logging.

Explain backup, replication, snapshots, live migration and disaster recovery.

Define Recovery Time Objective (RTO) and Recovery Point Objective (RPO), and propose suitable targets for at least two university applications.

Evidence Expected: A risk-and-mitigation table and a disaster-recovery plan with justified RTO and RPO values.

TASK 8 – Create the Virtualized Cloud Data Centre Architecture (5 Marks)

Construct an end-to-end architecture integrating compute, storage, network, virtualization and management components.

Draw a labelled architecture showing users, internet or campus network, firewall, load balancer, physical hosts, hypervisor layer, VMs/containers, virtual network, storage and monitoring.

Show how the university applications are distributed across hosts and how shared resources are allocated.

Include a management plane for provisioning, monitoring, logging and policy enforcement.

Show backup or replication to a secondary site or cloud region.

Justify the architecture in terms of scalability, performance, fault tolerance, security and cost.

Evidence Expected: A complete, clearly labelled architecture diagram and a written justification for each major component and design decision.

TASK 9 – Hands-on Implementation and Validation (5 Marks)

Build a small proof-of-concept that demonstrates virtualization and resource management.

Create at least one VM using VirtualBox, VMware, Hyper-V or KVM, or create a container using Docker.

Deploy a simple university-related service, such as a web page, student-record API or database-backed application.

Configure CPU, memory and storage resources and record the selected values.

Perform functional tests and at least one resource or load test. Record the observations.
Capture screenshots of the VM/container configuration, running service, resource-monitoring output and test results.
Document installation steps, commands, configuration details, issues faced and how they were resolved.
Evidence Expected: Reproducible implementation steps, source/configuration files, annotated screenshots and actual test evidence. Do not present simulated results as actual execution.

TASK 10 – Evaluate Cost, Performance and Final Recommendation (3 Marks)

Evaluate the proposed solution against the existing physical data-centre arrangement.
Compare the current physical-server model and proposed virtualized model in terms of hardware cost, electricity, cooling, maintenance, licensing and administration effort.
Prepare a simple three-year cost comparison using clearly stated assumptions.
Separate one-time capital expenditure from recurring operational expenditure.
Evaluate expected improvements in utilization, scalability, availability and manageability.
Identify risks such as licensing costs, vendor lock-in, skills requirements, hardware failure and migration downtime.
Provide a phased migration roadmap, identifying which applications should be migrated first and why.
Evidence Expected: A three-year cost table, risk analysis, migration roadmap and evidence-based final recommendation.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Virtualized Infrastructure (8 Marks)

Changed Requirement

The university introduces a digital learning platform and extends selected services to 10 affiliated colleges. The expected peak workload increases to five times the current peak. The system must achieve 99.9% availability, provide secure isolation between colleges and remain within a limited operating budget.
Re-evaluate your original design. You may RETAIN, MODIFY or REDESIGN the proposed infrastructure.

Impact Analysis – 2 Marks

Identify changes in workload, users, storage, network traffic and availability requirements.
Explain how multi-tenant access affects VM/container isolation, network segmentation and access control.
Identify which original capacity and cost assumptions are no longer valid.

Capacity and Availability Evaluation – 2 Marks

Estimate the additional CPU, memory, storage and network capacity needed for the five-times peak workload.

Evaluate whether the proposed number of hosts and the existing failover arrangement are sufficient.

Explain the meaning of 99.9% availability (approximately 8.76 hours of downtime per year) and identify measures needed to approach it.

Adaptation or Redesign – 2 Marks

Decide whether to retain, modify or redesign the original infrastructure and justify the decision.

Describe the revised architecture. Consider clustering, live migration, auto-scaling, container orchestration, multi-zone deployment or hybrid-cloud capacity where appropriate.

Explain changes to resource-allocation policies, tenant isolation, backup and disaster recovery.

Evidence-Based Defence – 2 Marks

Support the revised design with capacity calculations and clearly stated assumptions.

Compare the cost and operational complexity of the original and revised designs.

Discuss trade-offs involving performance, availability, security, licensing and management effort.

Evidence Expected: A technically justified decision supported by revised architecture, capacity calculations, availability considerations, security controls and cost comparison.

.Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks

Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load

- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks

Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)
- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks

Discuss:

- Capacity and availability calculations
- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

MARK DISTRIBUTION

Task	Title	Bloom's Level	Marks
Task 1	Existing Data Centre and Requirements	L2 – Understand / Analyse	3
Task 2	Virtualization Fundamentals and Hypervisors	L2 – Understand	3
Task 3	Server Consolidation and Resource Calculations	L3 – Apply	3
Task 4	Virtual Machines and Containers	L4 – Analyse	5
Task 5	Virtualized Storage and Network Infrastructure	L3/L4 – Apply / Analyse	4
Task 6	Resource Allocation, Scheduling and Monitoring	L4 – Analyse	5

Task	Title	Bloom's Level	Marks
Task 7	Availability, Security and Disaster Recovery	L4 – Analyse	3
Task 8	Virtualized Cloud Data Centre Architecture	L6 – Create	5
Task 9	Hands-on Implementation and Validation	L3 – Apply	4
Task 10	Cost, Performance and Final Recommendation	L5 – Evaluate	7
	Subtotal		

Task 11	Day-4 Design Challenge	L4/L5/L6 – Analyse / Evaluate / Create	8
	TOTAL		50

TASK 1 – ANALYSE THE EXISTING DATA CENTRE AND IDENTIFY REQUIREMENTS

1.1 Purpose and Major Components of a Data Centre

The main purpose of a data centre is to provide a reliable computing environment for hosting university applications such as the Student Information System (SIS), Learning Management System (LMS), Library Management System (LMS), Examination Portal and institutional website.

Component	Description
Compute	Physical servers containing CPUs and RAM to execute applications and process data.
Storage	HDDs, SSDs and storage systems used to store student records, examination results and application data.
Networking	Switches, routers and firewalls that provide communication between servers, users and the internet.
Power	UPS systems, power distribution units and generators that provide continuous electrical supply.
Cooling	Air-conditioning and cooling systems that maintain suitable operating temperatures for servers.
Management	Software used for server monitoring, configuration, resource allocation, security and maintenance.

1.2 Limitations of the Existing Physical Server Environment

1. **Low resource utilization:** Average CPU utilization is only 20%, meaning that most computing resources remain unused during normal periods.
2. **High operational cost:** Maintaining 20 separate physical servers increases electricity consumption, cooling expenses and hardware maintenance costs.
3. **Limited scalability:** Increasing computing capacity requires purchasing and installing new physical servers, which takes considerable time.
4. **Poor availability:** Failure of a physical server may interrupt the application running on it because there is no centralized VM management or tested live migration.
5. **Inadequate disaster recovery:** The university performs periodic backups but does not have a separate disaster recovery site or documented recovery objectives.

6. **Resource imbalance:** Individual applications cannot efficiently share the unused CPU and memory capacity of other physical servers.

1.3 Important Definitions

Term	Definition
Resource Utilization	The percentage of available CPU, memory, storage or network resources being used by applications.
Server Consolidation	Combining workloads from multiple physical servers onto fewer physical servers using virtualization.
Scalability	The ability of a system to increase or decrease its computing resources according to workload requirements.
Availability	The percentage of time a system remains operational and accessible to users.
Fault Tolerance	The ability of a system to continue operating even when one or more components fail.

1.4 Requirement Analysis Table

Requirement	Existing Problem	Proposed Solution
Performance	CPU utilization reaches 90–95% during peak periods.	Dynamic resource allocation, load balancing and CPU scheduling.
Capacity	20 physical servers are underutilized during normal periods.	Server consolidation using virtual machines and containers.
Security	Multiple applications require secure access to sensitive student data.	Network segmentation, firewalls, VM isolation and role-based access control.
Availability	Server failure can interrupt university applications.	High availability clustering, live migration and redundant hardware.
Cost	High electricity, cooling and maintenance costs.	Reduce physical server count through virtualization and shared resources.
Disaster Recovery	No separate disaster recovery site is available.	Automated backups, replication and a secondary recovery site.
Scalability	Hardware procurement is slow during periods of increased demand.	Virtualized infrastructure with dynamic resource allocation and cloud bursting.

TASK 2 – EXPLAIN VIRTUALIZATION FUNDAMENTALS AND HYPERVISORS

2.1 Purpose of Virtualization in a Cloud Data Centre

1. Server consolidation: Multiple applications can run on a single physical server.
2. Resource utilization: Unused CPU and memory resources can be allocated to other applications.
3. Scalability: Virtual machines can be created, modified or removed according to demand.
4. Isolation: Each virtual machine operates independently from other virtual machines.
5. Cost reduction: Fewer physical servers reduce electricity, cooling and maintenance expenses.
6. High availability: Virtual machines can be migrated to another host when a physical server fails or requires maintenance.

2.2 Role and Working of a Hypervisor

A hypervisor is a software layer that creates, manages and runs virtual machines on physical hardware.

It allocates CPU, memory, storage and network resources to virtual machines and ensures that multiple VMs can operate independently.

Working of a Hypervisor

1. The hypervisor accesses the physical hardware resources.
2. It creates virtual hardware for each virtual machine.
3. Each VM installs and runs its own guest operating system.
4. The hypervisor allocates resources to the VMs according to their requirements.
5. It manages resource sharing and isolation between virtual machines.

2.3 Virtualization Architecture Diagram

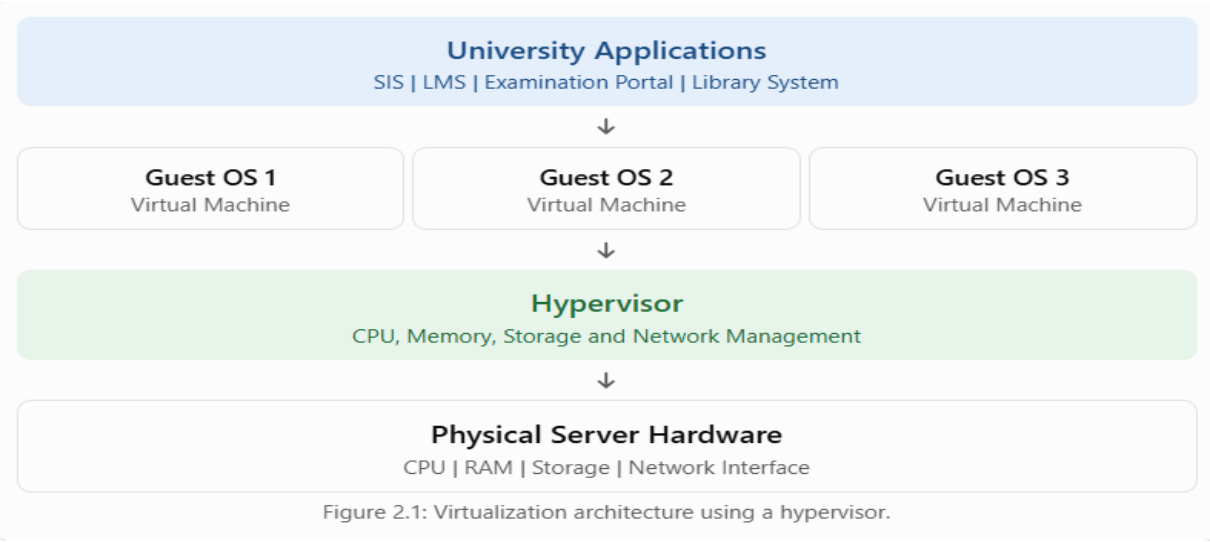


Figure 2.1: Virtualization architecture using a hypervisor.

2.4 Types of Hypervisors

Hypervisors are classified into two major types.

Feature	Type-1 Hypervisor	Type-2 Hypervisor
Name	Bare-metal hypervisor	Hosted hypervisor
Installation	Runs directly on physical hardware.	Runs on top of a host operating system.
Performance	High performance with low overhead.	Comparatively higher overhead.
Security	Smaller host OS dependency.	Depends on host OS security.
Usage	Enterprise data centres and cloud infrastructure.	Personal computers, testing and development.
Examples	VMware ESXi, Microsoft Hyper-V, KVM	Oracle VirtualBox, VMware Workstation

2.5 Comparison of Virtualization Technologies

Technology	Description	Example
Full Virtualization	Provides complete virtual hardware so that an unmodified guest OS can run.	VMware ESXi
Para-Virtualization	Guest OS is modified or uses special interfaces to communicate efficiently with the hypervisor.	Xen PV
Hardware-Assisted Virtualization	Uses processor features such as Intel VT-x and AMD-V to support virtualization.	KVM, Hyper-V
OS-Level	Multiple isolated containers share the same host	Docker

Virtualization	operating-system kernel.	

2.6 Recommended Hypervisor for the University

The university should use a Type-1 hypervisor such as VMware ESXi or KVM.

KVM is recommended as a cost-effective option because:

- It is an open-source virtualization technology.
- It supports hardware-assisted virtualization.
- It provides efficient CPU and memory management.
- It supports live migration and high availability when integrated with suitable management tools.
- It reduces dependence on expensive proprietary virtualization licensing.

The university can deploy KVM on multiple physical hosts and use a centralized management platform to create, monitor and migrate virtual machines.

TASK 3 – DESIGN SERVER CONSOLIDATION AND CALCULATE RESOURCE REQUIREMENTS

3.1 Physical-to-Virtual (P2V) Migration

Physical-to-Virtual (P2V) migration is the process of converting a workload running on a physical server into a virtual machine.

The process involves:

1. Assessing the physical server's CPU, memory, storage and application requirements.
2. Creating a virtual machine with suitable resources.
3. Migrating the operating system, applications and data into the VM.
4. Testing the migrated application.
5. Moving the application into production after successful validation.

P2V migration allows the university to reduce physical server count without completely redesigning its existing applications.

3.2 Resource Requirement Calculation

Given data

Parameter	Value
Number of existing physical servers	20
CPU cores per server	16 cores
RAM per server	64 GB

Storage per server	2 TB
Average CPU utilization	20%
Peak CPU utilization	90–95%
Target average CPU utilization	65%

Step 1: Calculate total physical CPU capacity

Total CPU capacity:

$$20 \times 16 = 320 \text{ cores}$$

Step 2: Calculate average CPU demand

$$\text{Average CPU Demand} = 320 \times \frac{20}{100}$$

$$\boxed{64 \text{ CPU cores}}$$

Step 3: Calculate peak CPU demand

The peak CPU utilization reaches 90–95%.

At 90%:

$$320 \times 0.90 = 288 \text{ cores}$$

At 95%:

$$320 \times 0.95 = 304 \text{ cores}$$

$$\boxed{\text{Peak CPU demand} = 288\text{–}304 \text{ cores}}$$

Step 4: Calculate total memory capacity

$$20 \times 64 = 1280 \text{ GB}$$

$$\text{Average Memory Demand} = 1280 \times 0.50$$

$$\boxed{640 \text{ GB}}$$

3.3 Calculate the Minimum Number of Hosts

Parameter	Proposed host specification
CPU	32 cores
RAM	256 GB
Storage	Shared storage system
Target average CPU utilization	65%
Capacity reserve	One host for N+1 failure protection

Step 1: Calculate the number of hosts based on CPU

$$N = \frac{\text{Total CPU demand}}{\text{CPU cores per host} \times \text{Target utilization}}$$

$$N = \frac{64}{32 \times 0.65}$$

$$\boxed{N = 4 \text{ hosts}}$$

Step 2: Check memory capacity

$$256 \times 0.65 = 166.4 \text{ GB}$$

$$N = \frac{640}{166.4}$$

$$\boxed{N = 4 \text{ hosts}}$$

Step 3: Add capacity reserve for host failure

Minimum hosts = 4.

Additional reserve host = 1.

$$\boxed{\text{Recommended configuration} = 5 \text{ hosts}}$$

3.4 Storage Requirement Calculation

$$20 \times 2 = 40 \text{ TB}$$

3.5 Resource Management Techniques

Technique	Explanation	University Application
Resource Overcommitment	Allocating more virtual CPU or memory than the physical capacity, based on expected usage.	Useful for low-utilization applications, but requires careful monitoring.
CPU Scheduling	The hypervisor distributes CPU execution time among virtual machines.	Prioritize examination and student-information applications during peak periods.
Memory Ballooning	The hypervisor reclaims unused memory from VMs through a balloon driver.	Redistributes memory to VMs experiencing higher demand.
Noisy-Neighbour Problem	One VM consumes excessive resources and affects the performance of other VMs.	Apply CPU limits, memory reservations and workload isolation.
Live Migration	Moves a running VM from one host	Supports maintenance and load

	to another with minimal downtime.	balancing.

TASK 4 – COMPARE VIRTUAL MACHINES AND CONTAINERS

4.1 Architecture of Virtual Machines

A virtual machine is a software-based computer with virtual CPU, memory, disk and network interfaces. Each VM runs its own guest operating system on a hypervisor. VMs provide strong separation between operating-system environments and are useful for legacy or different-OS applications.

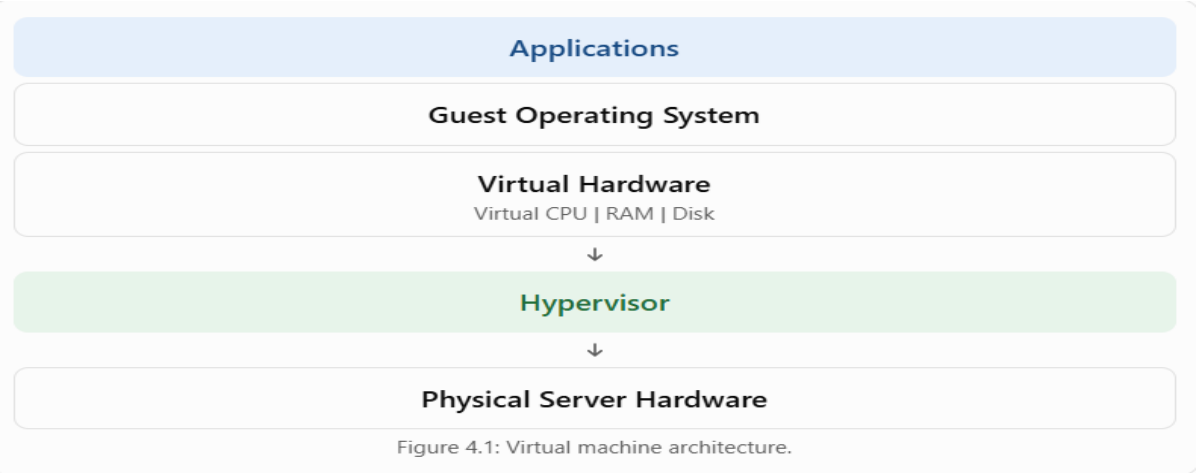


Figure 4.1: Virtual machine architecture.

Characteristics of VMs

- Each VM runs its own guest operating system.
- Provides strong isolation between virtual machines.
- Supports different operating systems on the same physical server.
- Requires more CPU, memory and storage overhead.
- Suitable for enterprise applications and legacy workloads

4.2 Architecture of Containers

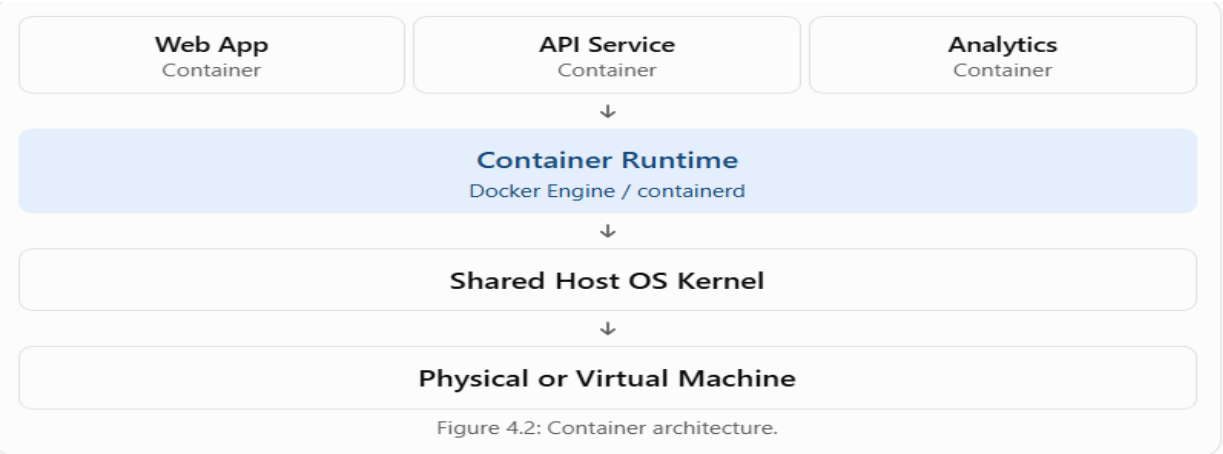


Figure 4.2: Container architecture.

Characteristics of Containers

- Containers share the host OS kernel.
- They are lightweight and require fewer resources.
- They start quickly compared with VMs.
- They provide application-level isolation.
- They are portable across compatible environments.
- They are suitable for microservices and cloud-native applications.

4.3 Comparison Between Virtual Machines and Containers

Feature	Virtual Machines	Containers
Virtualization	Hardware-level virtualization	OS-level virtualization
Operating System	Each VM has its own guest OS	Shares the host OS kernel
Isolation	Generally stronger isolation	Process-level isolation
Startup Time	Usually seconds to minutes	Usually milliseconds to seconds
Resource Overhead	Higher due to guest OS	Lower due to shared kernel
Portability	Portable as VM images	Highly portable as container images
Management	Managed using hypervisors	Managed using container runtimes and orchestrators
Performance	Additional virtualization overhead	Near-native performance for many workloads
Security	Separate guest OS provides an isolation boundary	Shared kernel requires additional security controls
Examples	VMware ESXi, KVM, Hyper-V	Docker, containerd, Kubernetes

4.4 Mapping University Workloads to VMs and Containers

University Workload	Recommended Technology	Justification
Student Information System (SIS)	VM	Legacy applications may require a dedicated guest OS and stable environment.

University Database	VM	Provides strong isolation and supports dedicated CPU, memory and storage resources.
Institutional Website	Container	Lightweight web services can be deployed and scaled quickly.
Learning Management System (LMS)	Containers	Web and application components can scale horizontally during peak demand.
Examination Portal	Containers	Multiple replicas can handle admission and examination traffic.

4.5 Role of Docker and Kubernetes

Docker

- Creating container images.
- Running applications in isolated containers.
- Providing consistent environments across development and production.
- Simplifying application deployment and portability.

Kubernetes

- Automatic container deployment.
- Horizontal scaling of application replicas.
- Load balancing across containers.
- Restarting failed containers.
- Rolling updates and application recovery.

4.6 Situations Where VMs Are More Appropriate

- Applications requiring a different operating-system kernel.
- Legacy applications that cannot be containerized easily.
- Applications requiring strong isolation between workloads.
- Databases requiring dedicated resource reservations.
- Applications requiring custom operating-system configurations.

TASK 5 – DESIGN VIRTUALIZED STORAGE AND NETWORK INFRASTRUCTURE

5.1 Storage Virtualization

Storage virtualization is the process of combining multiple physical storage devices into a single logical storage resource.

It allows virtual machines to access storage without depending on the location of the physical storage devices.

For example, the university can combine SSDs and HDDs from multiple storage devices into a shared storage pool and allocate virtual disks to different VMs.

5.2 Comparison of DAS, NAS and SAN

Feature	**DAS**	**NAS**	**SAN**
Full Form	Direct-Attached Storage	Network-Attached Storage	Storage Area Network
Connection	Directly connected to a server	Connected through a network	Dedicated storage network
Access Type	Block-level	File-level	Block-level
Sharing	Limited sharing	Multiple clients can share files	Multiple servers can access shared block storage
Performance	High for local workloads	Depends on network performance	High performance with suitable infrastructure
Cost	Low	Moderate	Higher
Example	Local HDD or SSD	Network file server	Fibre Channel or iSCSI storage
University Use	Local OS disks	Shared files and documents	Shared VM disks and databases

5.3 Software-Defined Storage (SDS)

Technology	**Description**	**University Application**
Thin Provisioning	Allocates logical storage capacity without immediately reserving the entire physical space.	Reduces storage wastage when creating VM disks.
Snapshots	Captures the state of a storage volume at a particular point in time.	Allows recovery from configuration errors.
Replication	Copies data to another storage system or location.	Supports disaster recovery.
Backup	Creates a separate recoverable copy of data.	Protects student records and

		examination results.
Storage Pooling	Combines multiple storage devices into a shared logical pool.	Simplifies storage allocation across VMs.

5.4 Network Virtualization

Network virtualization is the process of creating logical networks using software over a shared physical network infrastructure.

It allows the university to create separate networks for student services, databases, administration and management without requiring separate physical switches for each network.

Component	Description
Virtual Switch	Connects VMs and containers within a virtualized host.
VLAN	Creates logically isolated Layer 2 networks using VLAN identifiers.
Virtual Network	A software-defined logical network connecting virtual resources.
SDN	Separates network control functions from packet forwarding to enable centralized network management.
Virtual Router	Routes traffic between logical networks.
Virtual Firewall	Applies security policies to network traffic between virtual resources.

5.5 Proposed University Network Segmentation

Network Segment	Purpose	Security Control
Administration Network	Faculty and administrative applications	Restricted access through firewall rules
Student Services Network	SIS, LMS and student portals	Controlled access from campus and internet
Database Network	Student records and examination databases	Only authorized application servers can access
Management Network	Hypervisor management, monitoring and configuration	Restricted administrator access with MFA
Backup Network	Backup transfer and replication	Isolated from normal

		application traffic

TASK 6: PLAN RESOURCE ALLOCATION, SCHEDULING AND MONITORING

6.1 Resource Allocation Policy

Resource	Allocation policy
CPU	Assign virtual CPUs (vCPUs) based on workload demand. Use CPU shares to prioritize important applications during contention.
Memory	Allocate sufficient RAM to each VM or container. Set memory reservations for critical services and limits to prevent excessive consumption.
Storage	Allocate virtual disks or container volumes according to application requirements. Monitor capacity and disk I/O performance.
Network	Assign virtual network interfaces and bandwidth policies. Use network segmentation to separate student, administration, database and management traffic.

6.2 Static and Dynamic Allocation

Method	Description	Use
Static allocation	Resources are assigned in advance and remain fixed unless manually changed.	Suitable for stable, predictable workloads such as core databases.
Dynamic allocation	Resources are adjusted according to demand or policies.	Suitable for variable workloads such as admission portals and online learning systems.

6.3 Scheduling, Load Balancing and High Availability

- Workload scheduling: Places VMs or containers on hosts with sufficient available CPU, memory and storage. Scheduling should avoid overloading one host.
- Load balancing: Distributes incoming requests across multiple instances of an application. This prevents a single VM or server from becoming a bottleneck.
- Auto-scaling: Adds or removes VM or container instances according to workload demand. During admission periods, additional application instances can be started; during quiet periods, unnecessary instances can be removed.

- High availability (HA): Restarts affected VMs or services on another available host when a physical host fails. Critical applications should run across separate hosts to reduce the risk of a single-host failure.

6.4 Monitoring Indicators and Alerting Plan

Indicator	Example alert threshold	Corrective action
CPU utilization	Warning above 75%; critical above 90% for a sustained period	Review CPU contention, rebalance workloads, increase allocated vCPU or add capacity.
Memory utilization	Warning above 80%; critical above 90%	Identify memory-heavy workloads, check for memory pressure, adjust allocation or add RAM.
Disk I/O	Sustained high I/O utilization or increasing I/O wait	Identify storage-intensive workloads, distribute I/O, or move data to faster storage.
Network throughput	Sustained use above 80% of available bandwidth	Check traffic patterns, adjust bandwidth allocation or increase network capacity.
Application latency	Response time exceeds the application's agreed service target	Check CPU, memory, storage and network bottlenecks; scale or rebalance the service.
VM availability	VM or service becomes unreachable, or availability falls below its target	Trigger HA recovery, investigate the host or service failure, and verify recovery.

6.5 Monitoring and Alerting Methods

A centralized monitoring platform should collect metrics from physical hosts, hypervisors, VMs, containers, storage and virtual networks. Dashboards should show current utilization, historical trends and capacity forecasts.

Alerts can be sent to data-centre administrators through email, SMS or an operations dashboard. Alerts should include the affected host or service, metric, threshold, severity and time of occurrence.

6.6 Administrator Response to Resource Contention

1. Identify the affected host and resource using monitoring dashboards.
2. Determine which VM or container is consuming excessive CPU, memory, storage I/O or network bandwidth.
3. Check whether the workload is experiencing a temporary peak or a sustained increase in demand.
4. Rebalance workloads, adjust reservations, limits or shares, or migrate a VM to a less-loaded host where supported.
5. Scale out application instances or add physical/cloud capacity if the demand is sustained.
6. Verify that performance and availability have returned to acceptable levels, and record the incident for future capacity planning.

TASK 7 – EVALUATE DATA CENTRE AVAILABILITY, SECURITY AND DISASTER RECOVERY

7.1 Virtualization Risks and Controls

Risk	Description	Recommended control
Hypervisor attack	An attacker compromises the hypervisor and may affect multiple VMs.	Use a hardened hypervisor, apply security patches, restrict management access and monitor logs.
VM escape	Malicious code inside a VM attempts to access the host or other VMs.	Keep hypervisor software updated, use strong VM isolation and apply least-privilege access.
VM-to-VM attack	A compromised VM attempts to attack another VM on the same host or network.	Use virtual network segmentation, firewall rules and access controls between workloads.
Insecure VM images	Unpatched or untrusted VM/container images may contain vulnerabilities or malware.	Use approved images, scan them for vulnerabilities and update them regularly.
Insecure snapshots	Snapshots may expose sensitive data or remain accessible longer than required.	Restrict snapshot access, protect snapshot storage and remove unnecessary snapshots.
VM sprawl	Uncontrolled creation of VMs	Maintain a VM inventory, require

	increases cost and creates unmanaged security risks.	approval and remove unused VMs.
Denial of Service (DoS)	One workload consumes excessive CPU, memory or network resources, affecting others.	Apply resource limits, monitor utilization and isolate workloads where necessary.

7.2 Security Measures

- Network segmentation: Separate administration, student, database and management networks using VLANs or virtual networks and firewall rules.
- Encryption: Encrypt sensitive data at rest and in transit.
- Multi-factor authentication (MFA): Require MFA for administrators accessing the virtualization management platform.
- Role-based access control (RBAC): Give users only the permissions needed for their responsibilities.
- Patching and updates: Regularly patch hypervisors, VM operating systems, containers and applications.
- Audit and monitoring: Record administrative actions, access attempts and security events; review logs and alerts regularly.
- Image security: Use trusted, approved VM and container images and scan them before deployment.

7.3 Backup, Replication and Disaster Recovery

Control	Purpose
Backup	Creates recoverable copies of VM data, application data and configurations.
Replication	Copies data or workloads to another system or site to reduce recovery time after a failure.
Snapshots	Capture a VM's state at a particular point in time; useful for short-term rollback, but not a replacement for independent backups.
Live migration	Moves a running VM between compatible hosts with minimal service interruption, where supported.
Disaster	Provides an alternate environment for restoring important services after a

recovery site	major data-centre failure.

Backups should be stored separately from the primary environment, access should be restricted, and restore tests should be performed periodically. The university should document recovery procedures and assign responsibility for executing them.

7.4 Example RTO and RPO Targets

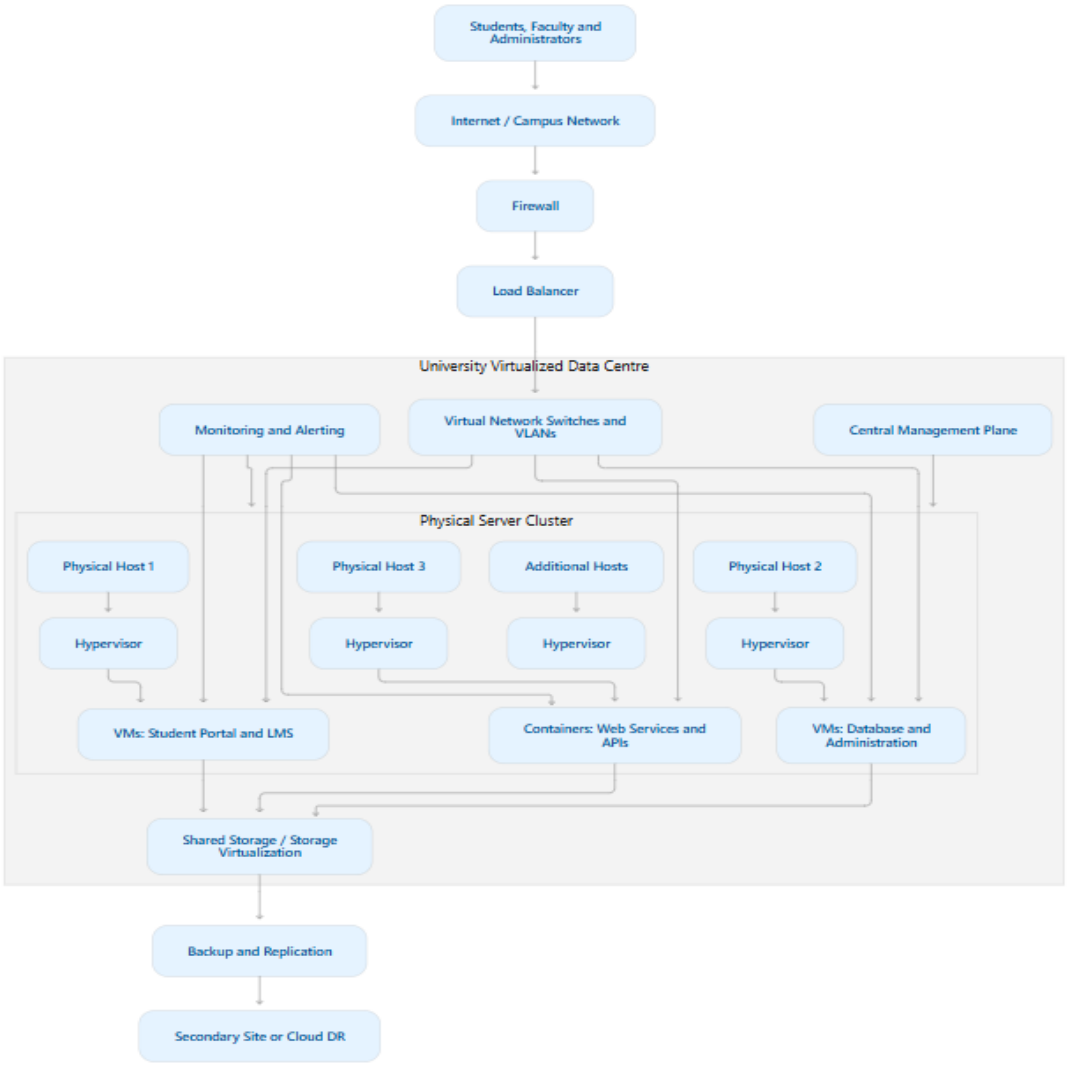
The assignment scenario does not provide existing Recovery Time Objectives (RTOs) or Recovery Point Objectives (RPOs). The following are suggested targets for planning and should be approved by the university.

Application	Suggested RTO	Suggested RPO
Student information system	4 hours	1 hour
Learning management system (LMS)	2 hours	30 minutes

- RTO (Recovery Time Objective): Maximum acceptable time to restore an application after an outage.
- RPO (Recovery Point Objective): Maximum acceptable amount of data loss measured in time.

TASK 8 – CREATE THE VIRTUALIZED CLOUD DATA CENTRE ARCHITECTURE

8.1 Architecture Diagram



8.2 Architecture Components and Functions

Component	Function
Users and campus/Internet network	Provide access for students, faculty and administrators to university applications.
Firewall	Filters incoming and outgoing traffic and blocks unauthorized access.
Load balancer	Distributes application requests across multiple service instances.
Physical server cluster	Provides CPU, memory and physical network resources for virtualization.
Hypervisor	Runs and manages multiple VMs on each physical host.
VMs and containers	Host university applications, databases, web services and APIs.

Virtual switches and VLANs	Connect virtual workloads and separate traffic into logical networks.
Shared storage	Provides centralized storage for VM disks and application data.
Central management plane	Allows administrators to manage hosts, VMs, resource allocation and migration from a central platform.
Monitoring and alerting	Tracks CPU, memory, storage, network performance and service availability.
Backup and replication	Protects data and supports recovery at a secondary site or cloud environment.

8.3 Network Segmentation

The virtual network should separate university workloads into logical segments:

- Student network: Access to student-facing portals and learning services.
- Administration network: Access to administrative applications and systems.
- Database network: Restricted access for application servers and authorized administrators.
- Management network: Access to hypervisors and virtualization management tools, limited to authorized IT staff.

Firewall rules should control communication between these segments. For example, student devices should not have direct access to the database network.

8.4 Workload Distribution and Availability

The university should distribute critical VMs and application instances across different physical hosts. This reduces the chance that a single host failure will interrupt all instances of a service.

- Load balancing across multiple application instances.
- Centralized resource allocation for CPU, memory, storage and network capacity.
- VM migration between compatible hosts to support maintenance and load balancing.
- High availability by restarting affected VMs on another available host after a host failure.
- Backup and replication to support recovery after data loss or a major site failure.

TASK 9 – HANDS-ON IMPLEMENTATION AND VALIDATION

9.1 aim

To develop and deploy a simple University Student Record REST API using Python Flask and Docker. The application is tested to verify its functionality and demonstrate containerization with resource allocation.

9.2 requirements and configuration

Software Requirements:

1. Python 3
2. Flask framework
3. Docker Desktop or Docker Engine
4. Notepad or Visual Studio Code
5. Web Browser or Command Prompt

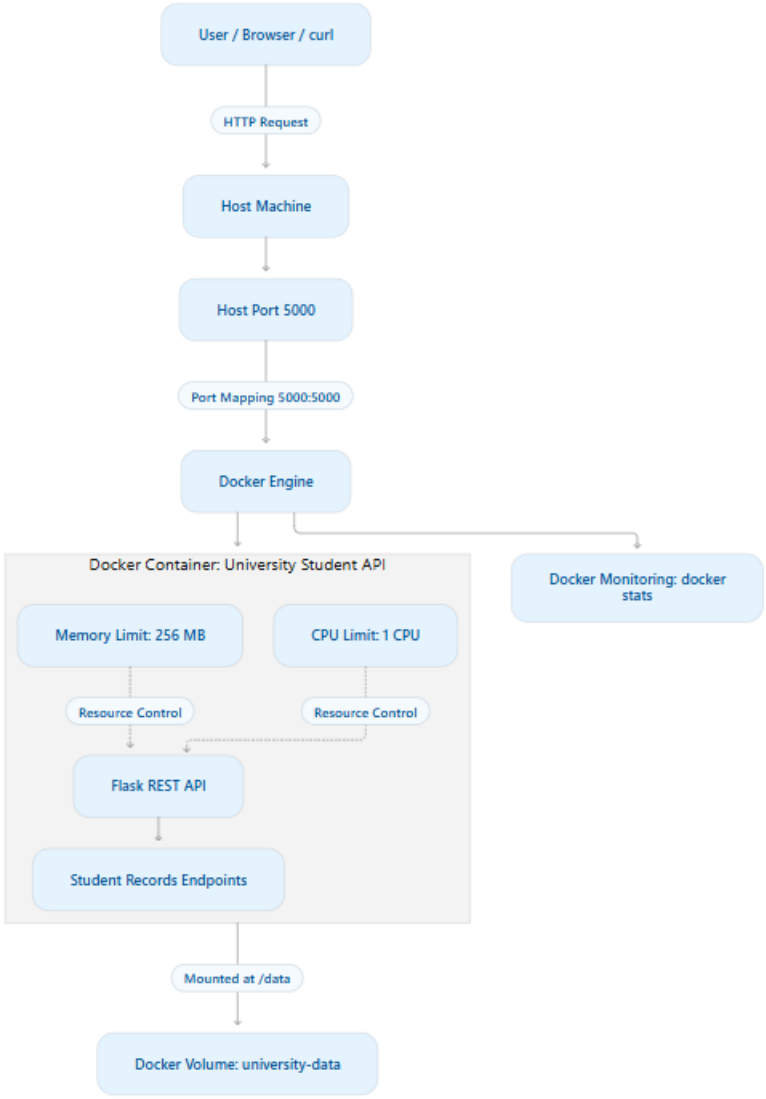
Resource Configuration:

Parameter	Configuration
CPU	1 CPU
Memory	512 MB
Port Number	5000
Programming Language	Python
Framework	Flask
Container Platform	Docker

The application provides sample student information through REST API endpoints.

9.3 proposed implementation architecture

Figure 1 – Docker-Based University Student API Architecture



9.4 program – app.py

Create a Python file named app.py and enter the following code.

```
from flask import Flask, jsonify

app = Flask(__name__)

students = [
    {"reg_no": "101", "name": "Arun", "department": "CSE"},
    {"reg_no": "102", "name": "Meena", "department": "CSE"}
]
```

```
@app.route("/")
def home():
    return "University Student API is running"

@app.route("/students")
def get_students():
    return jsonify(students)

@app.route("/students/<reg_no>")
def get_student(reg_no):
    for student in students:
        if student["reg_no"] == reg_no:
            return jsonify(student)
    return jsonify({"error": "Student not found"}), 404

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Program Explanation:

1. Flask is imported to create the REST API.
2. A list named students stores sample student details.
3. The / route displays the home page message.
4. The /students route displays all student records.
5. The /students/<reg_no> route searches for a particular student.
6. If the student is not found, the application returns a 404 error.

9.5 create requirements.txt

Create another file named requirements.txt and enter the following:

```
Flask==3.0.3
```

This file specifies the Flask version required to run the application.

9.6 create dockerfile

Create a file named Dockerfile without any file extension.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Dockerfile Explanation:

1. FROM selects the Python base image.
2. WORKDIR creates the working directory.
3. COPY copies the application files.
4. RUN installs Flask.
5. EXPOSE specifies port 5000.
6. CMD starts the Flask application.

9.7 build and run commands

Save all three files in the same folder:

1. app.py
2. requirements.txt
3. Dockerfile

Open Command Prompt in that folder and execute the following commands.

Step 1: Build the Docker Image

docker build -t university-student-api .

This command creates a Docker image named university-student-api.

Step 2: Run the Docker Container

docker run -d --name student-api -p 5000:5000 --cpus="1.0" --memory="512m" university-student-api

This command starts the container with one CPU and 512 MB memory limit.

Step 3: Check the Running Container

docker ps

This command displays the running Docker containers.

Step 4: Monitor Resource Usage

docker stats student-api

This command displays the CPU and memory usage of the container.

9.8 testing the application

Open a web browser and enter the following URLs.

Test Case 1: Home Page

URL:

http://localhost:5000/

Expected Output:

University Student API is running

Test Case 2: Display All Student Records

URL:

http://localhost:5000/students

Expected Output:


```
[
  {
    "department": "CSE",
    "name": "Arun",
    "reg_no": "101"
  },
  {
    "department": "CSE",
    "name": "Meena",
```

"reg_no": "102"
}
]

Test Case 3: Search for an Existing Student

URL:

http://localhost:5000/students/101

Expected Output:

{
"department": "CSE",
"name": "Arun",
"reg_no": "101"
}

Test Case 4: Search for a Missing Student

URL:

http://localhost:5000/students/999

Expected Output:

{
"error": "Student not found"
}

HTTP Status Code: 404

9.9 resource monitoring and validation

The Docker container is configured with a CPU limit of 1 CPU and a memory limit of 512 MB.

The following command is used to monitor the container's resource usage.

docker stats student-api

The command displays:

1. CPU usage.
2. Memory usage.
3. Network input and output.
4. Container resource statistics.

The application is validated by checking the following conditions:

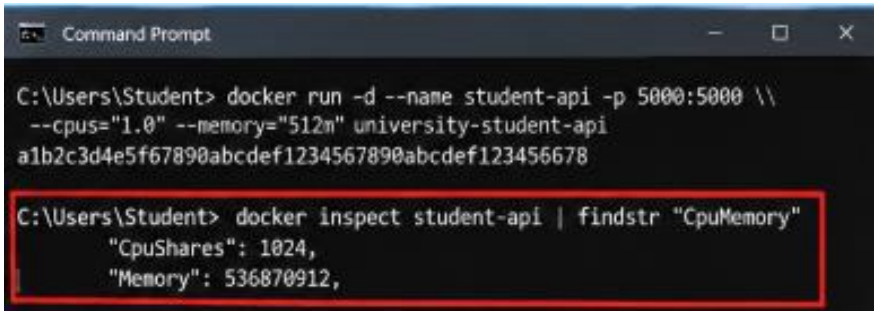
1. Docker container starts successfully.
2. Student records are displayed correctly.
3. Individual student details are retrieved.
4. Missing student requests return HTTP 404.
5. CPU and memory usage can be monitored.

9.10 screenshots

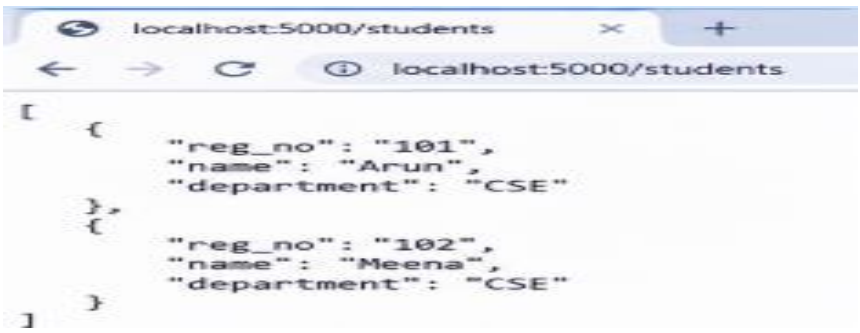
- Docker container running successfully using the docker ps command.



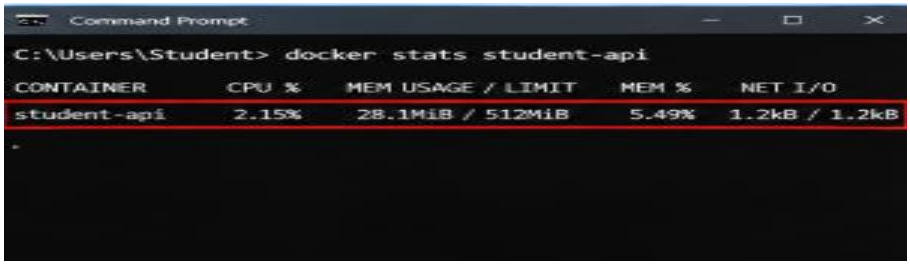
- Docker container configuration showing CPU limit of 1 CPU and memory limit of 512 MB.



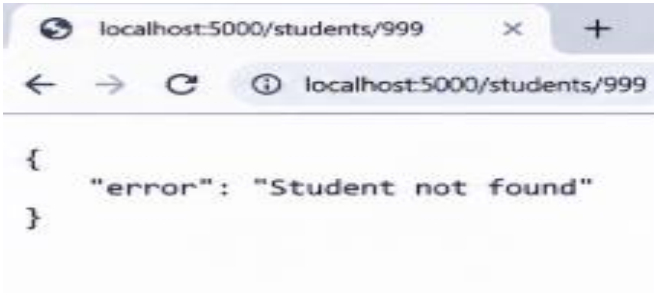
- Student records displayed in JSON format using the /students URL.



- CPU and memory usage displayed using the docker stats student-api command.



- Student-not-found error displayed using the /students/999 URL.



9.11 result

Thus, a University Student Record REST API was developed using Python Flask and deployed using Docker. The application was tested for retrieving all student records, searching for individual students, and handling missing student requests. CPU and memory limits were configured, and resource usage was monitored using Docker. Hence, the implementation demonstrates containerization and resource management in a cloud computing environment.

TASK 10 – EVALUATE COST, PERFORMANCE AND FINAL RECOMMENDATION

10.1 Cost Assumptions

The following is an illustrative planning estimate in Indian Rupees (INR), not a vendor quotation. Actual costs depend on hardware brands, support contracts, electricity tariffs, software licensing and procurement discounts. Assume the existing 20-server environment is replaced by a 16-host peak-capable virtualized cluster, based on Task 3 peak calculation. Amounts are in lakh rupees (₹ lakh).

Cost item	Existing physical model	Virtualized model
Initial hardware / storage / network CAPEX	₹80 lakh	₹100 lakh
Annual electricity and cooling	₹18 lakh	₹12 lakh
Annual maintenance and support	₹10 lakh	₹8 lakh
Annual licensing and management	₹4 lakh	₹8 lakh
Annual administration effort (estimated)	₹8 lakh	₹6 lakh

10.2 Three-Year Cost Comparison

Existing model recurring cost per year = 18 + 10 + 4 + 8 = ₹40 lakh

Existing 3-year TCO = CAPEX 80 + (40 × 3) = ₹200 lakh

Virtualized model recurring cost per year = 12 + 8 + 8 + 6 = ₹34 lakh

Virtualized 3-year TCO = CAPEX 100 + (34 × 3) = ₹202 lakh

Cost measure	Existing physical model	Virtualized model
One-time CAPEX	₹80 lakh	₹100 lakh
Annual OPEX	₹40 lakh	₹34 lakh
Three-year OPEX	₹120 lakh	₹102 lakh
Three-year total cost	₹200 lakh	₹202 lakh

Interpretation: Under these illustrative assumptions, the virtualized design costs ₹2 lakh more over three years because of higher initial infrastructure and licensing expenditure. However, its annual operating cost is ₹6 lakh lower. The simple payback on the additional ₹20 lakh CAPEX is about 20/6 = 3.33 years, so the financial case becomes stronger over a longer period or if utilization, outage reduction and deployment speed create additional savings. A detailed procurement quotation is required before approval.

10.3 Performance and Operational Comparison

Factor	Existing physical model	Proposed virtualized model
Utilization	Average CPU utilization	Pooled resources can be

	about 20%.	allocated to workloads according to demand.
Scalability	Requires purchase and installation of another physical server.	VMs and containers can be provisioned faster; cluster can be expanded.
Availability	No tested live migration or defined DR site.	Clustering, live migration, replication and tested DR plan.
Management	Separate server administration.	Centralized provisioning, monitoring and policy management.
Power and cooling	More separate servers and associated cooling.	Potential reduction through consolidation, subject to peak capacity.
Deployment	Hardware procurement delays.	Templates and container images support repeatable deployment.

10.4 Risks and Controls

Risk	Control
Virtualization licensing cost	Compare licensing models, open-source options and support costs before procurement.
Vendor lock-in	Use portable VM formats, standard APIs, container images and documented exit plans.
Skills gap	Train administrators in hypervisors, networking, storage, containers and security.
Hardware failure	Use N+1 capacity, redundant power/network paths and spare parts.
Migration downtime	Pilot low-risk systems, schedule maintenance windows, test rollback and verify data.

Performance contention	Use reservations, limits, shares, monitoring and capacity planning.

10.5 Phased Migration Roadmap

Phase	Activities	Reason
Phase 1 – Assessment	Inventory servers, dependencies, utilization, licenses, data and recovery needs.	Avoids migration surprises and identifies suitable workloads.
Phase 2 – Build foundation	Install hypervisor cluster, shared storage, redundant network, identity and monitoring.	Creates a secure platform before moving applications.
Phase 3 – Pilot	Migrate development, test and low-risk web services; validate performance and backup.	Tests the process with limited business risk.
Phase 4 – Core applications	Migrate library and institutional website, then SIS and LMS in planned windows.	Moves workloads in order of complexity and business criticality.
Phase 5 – Optimize	Tune resource allocations, remove unused servers, test failover and DR.	Confirms benefits and improves capacity utilization.

10.6 Final Recommendation

The university should adopt a phased virtualized private-cloud design using a Type-1 hypervisor cluster, shared storage, segmented virtual networking and containers for modern web services. The peak-capacity calculation indicates that a 16-host, 32-core design is needed to support the stated current peak at a 65% planning target with one-host failure reserve, under the assumptions in Task 3. Procurement should be staged and supported by workload measurements. The financial estimate shows that savings in annual operating cost may take

more than three years to recover the higher initial investment; therefore, licensing, support and hardware quotations must be compared before final approval.

TASK 11 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE INFRASTRUCTURE

Changed requirement: The university introduces a digital learning platform and extends selected services to affiliated colleges. Peak workload increases to five times the current peak. The service must achieve at least 99.9% availability, isolate tenants securely and remain within a limited operating budget. The question also refers to 10 affiliated colleges in the scenario and a 40-college isolation point in the later evidence prompts. The design below uses 40 tenants for the stricter isolation case; if the final scope is only 10 colleges, the same model applies with fewer tenant configurations.

11.1 Impact Analysis (2 Marks)

- **Workload:** Digital learning, video/content access, assessments and API traffic increase peak CPU demand to five times the current peak.

- **Users:** Students, faculty and administrators from the university and affiliated colleges access shared services concurrently.

- **Storage:** Learning content, submissions, attendance and assessment records increase storage and backup requirements.

- **Network:** More concurrent sessions, file downloads, video traffic and API calls increase bandwidth and load-balancer demand.

- **Availability:** A single host or site failure must not make the entire platform unavailable; redundant zones and tested recovery are needed.

- Original assumptions affected: The earlier host count, network capacity, storage growth, single-tenant access policies and cost estimate are no longer sufficient without revalidation.

- Web services and APIs become critical because the LMS, student systems, mobile applications and college portals need standardized, controlled communication. An API gateway can authenticate requests, apply rate limits, route traffic and provide logging.

- Multi-tenancy: Each college must have a unique tenant identity and logically separated data, roles, network policies and audit records. Tenant identifiers must be enforced by the application and database access layer, not merely supplied by the browser.

11.2 Capacity and Availability Evaluation (2 Marks)

Use the current peak CPU demand calculated in Task 3: 304 cores. Assume the new peak is five times this demand.

New peak CPU demand = 304 × 5 = 1,520 CPU cores

Host capacity = 32 physical cores; target utilization = 65%

Planned CPU demand per host = 32 × 0.65 = 20.8 cores

Hosts without failure reserve = 1,520 / 20.8 = 73.08

Round up = 74 hosts

With one-host failure reserve = 75 hosts

After one host fails: 74 × 32 × 0.65 = 1,539.2 cores

This is greater than 1,520 cores.

Memory estimate: Earlier assumed average memory demand was 768 GB. If peak memory is assumed to be 1.5 times average and then increases fivefold, estimated new peak memory = 768 × 1.5 × 5 = 5,760 GB. With 512 GB per host, 75 hosts provide 38,400 GB; after one host failure, 37,888 GB. Memory is therefore not the limiting factor under this simplified assumption. Actual memory use must be measured because workload growth may not scale linearly.

Storage and network: The question does not provide per-user storage or bandwidth figures, so an exact TB or Gbps value cannot be calculated honestly. Measure current peak storage growth, IOPS, throughput and concurrent sessions. For initial planning, reserve at least five times the measured peak workload capacity, plus separate backup/replication bandwidth and

storage growth margin. Use object storage/CDN for large static learning content where suitable.

Availability: 99.9% annual availability allows approximately 0.1% downtime. 365 × 24 × 0.001 = 8.76 hours per year. This target requires redundant hosts, redundant network paths and power, health checks, load balancing, automated recovery, multi-zone deployment, tested backups and clear maintenance procedures.

11.3 Adaptation or Redesign

Decision: MODIFY the original design into a hybrid, multi-zone, multi-tenant architecture. Retain the Type-1 virtualization foundation and shared storage principles, but add container orchestration, an API gateway, tenant-aware security, multi-zone deployment and cloud bursting for temporary peaks. A complete redesign is not necessary because the original virtualization base remains useful, but its capacity and availability design must be expanded.

11.4 Revised Service and Resource Policies

Use IaaS for controlled VM workloads and network foundations; use PaaS/managed database services where they reduce administration and support high availability; use containers and Kubernetes for scalable application services.

Use an API gateway for authentication, throttling, routing, versioning and audit logging.

Use separate namespaces, network policies, quotas and service identities for each tenant. For highly sensitive workloads, consider dedicated databases or dedicated node pools.

Use horizontal auto-scaling for stateless web/API services; scale databases carefully using tested read replicas, connection pooling and caching.

Set per-tenant CPU, memory, storage and request quotas to prevent one college from exhausting shared resources.

Replicate critical data across zones and maintain immutable backups in a separate region.

11.5 Evidence-Based Defence and Trade-offs

Area	Original design	Modified design
Peak CPU planning	16 hosts for current 304-core peak with N+1.	Approximately 75 hosts at five times peak under the same 32-core/65%

		assumptions, or use measured hybrid-cloud burst capacity.
Availability	Single cluster and secondary backup site proposed.	Multi-zone service deployment, redundant load balancers, automated recovery and secondary-region DR.
Tenant security	Basic VLAN and role separation.	Tenant-aware identity, data partitioning, namespace/network policies, quotas, per-tenant audit and optional dedicated resources.
Cost	Higher initial hardware cost but reduced annual OPEX estimate.	Higher variable cloud and managed-service costs during peaks; capacity reservations and autoscaling control waste.
Complexity	Conventional VM cluster administration.	More complex orchestration, API gateway, tenant governance and cloud operations.

Cost control strategy: Keep predictable baseline workloads on the university's private cluster; use auto-scaling and cloud bursting for short peak periods after performance and security testing. Apply budgets, quotas, tagging and cost alerts per college. Compare reserved capacity with on-demand pricing and avoid running peak capacity continuously when demand is seasonal.

Trade-offs: Multi-zone deployment improves resilience but increases infrastructure and network costs. Managed PaaS reduces operational effort but may increase vendor lock-in. Kubernetes supports scale and portability but requires trained administrators. Strong tenant isolation improves security but can reduce resource sharing efficiency. The university should balance these factors using measured load tests and a pilot with selected colleges.

Final decision: Modify the original design. Retain virtualization for the stable baseline, add container orchestration and API management for elastic digital learning services, and use multi-zone/hybrid-cloud capacity to handle five-times peak demand. The calculated 75-host figure is a theoretical all-on-premises peak plan; a hybrid approach can reduce capital expenditure if burst capacity is contractually available and tested. The 99.9% target must be verified through monitoring and failover exercises, not assumed from architecture alone.